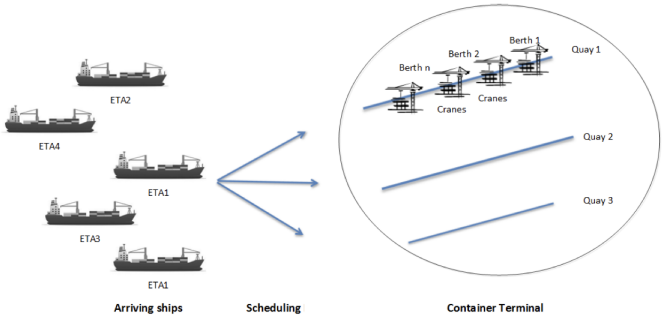

IASD 2024/25
Project Assignment #3:
Berth Allocation Problem

Luis Custódio and Rodrigo Ventura
Instituto Superior Técnico, University of Lisbon

(Version 1.0, October 13, 2024)

1 Introduction

The project for this assignment remains the same. The system receives as input information concerning a set of vessels that need berth space in a container terminal. The goal of the project is to allocate berth space to vessels and to schedule the vessels such that the total weighted flow time is minimized.

2 Third Assignment and Deliverable

This assignment is a followup of both Assignment #1, where the problem was properly formalized, and of Assignment #2, aiming at obtaining optimal solutions using uninformed search methods. This third and final assignment aims at obtaining optimal solutions to the

problem using **informed search methods**. As in the previous assignment, the informed algorithms that can be used are the ones available in the `search.py` from the AIMA repository[1]. Again, you only need to import `search.py`, which in turn imports `utils.py`. Both files are available in the repository. Copy both files to your work directory while developing and testing on your computer. Nothing else is needed.

The Python program to be delivered should be called `solution.py` and include (at least) a class with name `BAProblem` containing (at least) the methods required in Assignment #2 (see Annex A for a class template), as well as your heuristic function as the following class method:

> `h(state)` returns the heuristic of the given `state`.

In order to solve a problem one needs to:

- read the problem from a file object like you did for Assignment #1,
- you may change the state representation from Assignment #2 if you find a better one; in that case, methods `result`, `actions`, `goal_test`, and `path_cost` need to be adjusted accordingly; you may also change the implementation of `actions` and `result` methods, if for instance you have struggled with execution times in assignment #2.
- design and implement a heuristic function `h`; use docstrings (triple quotation marks) to describe and characterize in terms of optimality what your heuristic do.
- choose one (and only one) informed search algorithm that the group considers best fits the problem at hand, and use it in method `solve`. See `search.py` for the set of informed search algorithms available.

3 Evaluation

The deliverable for this assignment is shown through DEEC Moodle, with the submission of a single python file, called `solution.py`, implementing the modules mentioned above. Instructions for this platform are available on the course webpage. Finally, the grade is computed in the following way:

- 50% from the public tests;
- 50% from the private tests; and
- -10% from the code structure, quality and readability.

Deadline: **25-October-2024** (Lisbon time). Projects submitted after the deadline will not be considered for evaluation.

[1] The link for the repository is https://github.com/aimacode/aima-python.

Closing Remarks on Ethics:

- Any kind of sharing code outside your group is considered plagiarism;
- Developing your code in any public shared software development tool is considered sharing code;
- You can use GitHub, but make sure the project is marked private, and remove it afterward;
- If you get caught in any form of plagiarism, either by copying the code/ideas or sharing them with others, you will not be graded; and
- The scripts and other supporting materials produced by the instructors cannot be made public.

A Class Template

```python
import search

class BAProblem(search.Problem):

        def __init__(self):
                """Method that instantiates your class.
                     You can change the content of this.
                     self.initial is where the initial state of
                     the BAProblem should be saved."""
                self.initial = None

        def result(self, state, action):
                """Returns the state that results from executing
                the given action in the given state. """
                pass

        def actions(self, state):
                """Returns the list of actions that can be executed in
                the given state."""
                pass

        def goal_test(self, state):
                """Returns True if the state is a goal."""
                pass

        def path_cost(self, c, state1, action, state2):
                """Returns the cost of a solution path that arrives
                at state2 from state1 via action, assuming cost c
                to get up to state1."""
                pass

        def solve(self):
                """Calls the informed search algorithm chosen.
                Returns a solution using the specified format."""
                pass

        def load(self, fh):
                """Loads a BAP problem from the file object fh.
                     It may initialize self.initial here."""
                pass
```

```python
def h(self, node):
        """Returns the heuristic value of the state
        associated with node.
        This heuristic works like this:
        ...
        This heuristic does / does not guarantee optimality
        because..."""
        pass
```